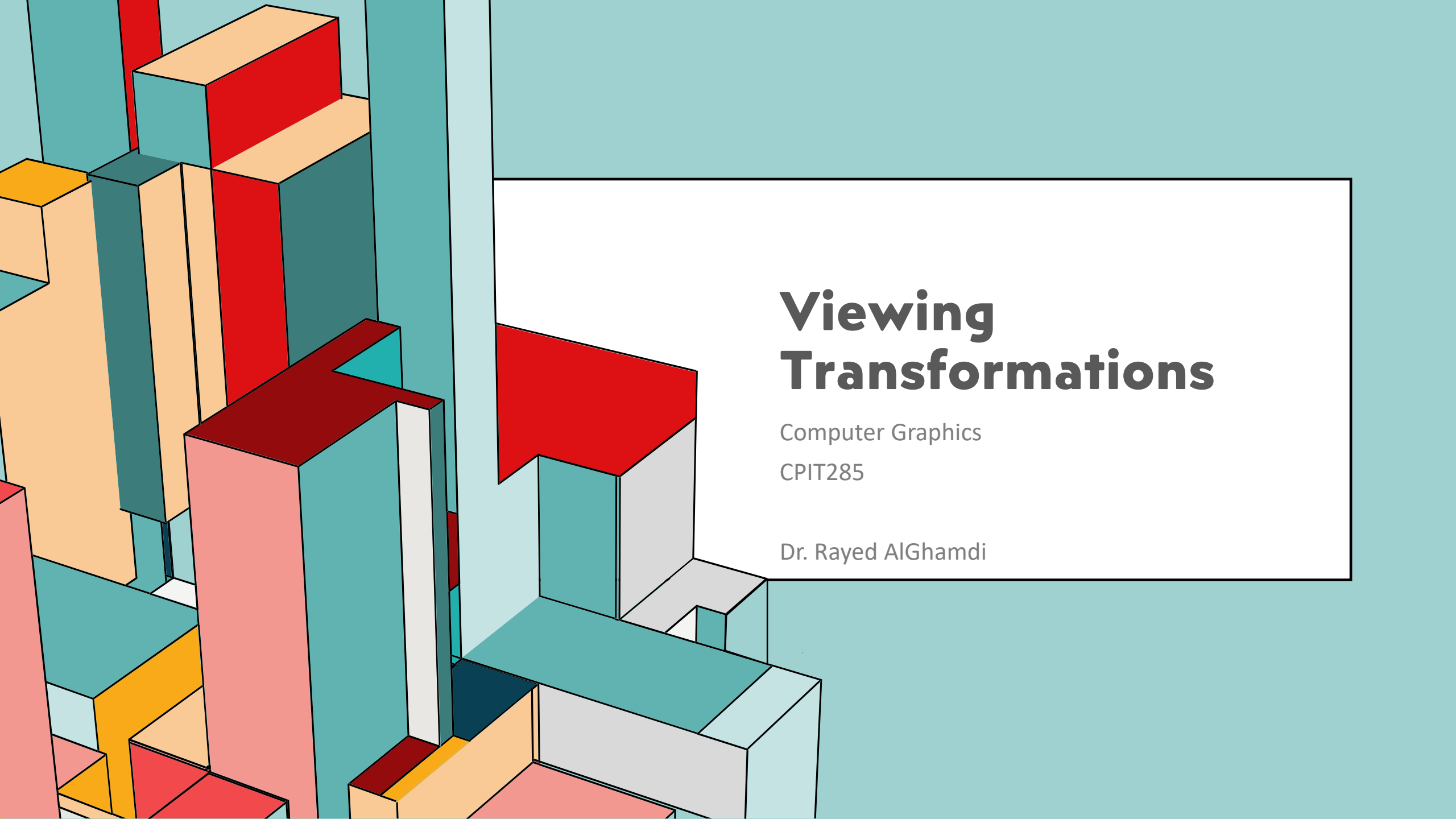




Viewing Transformations

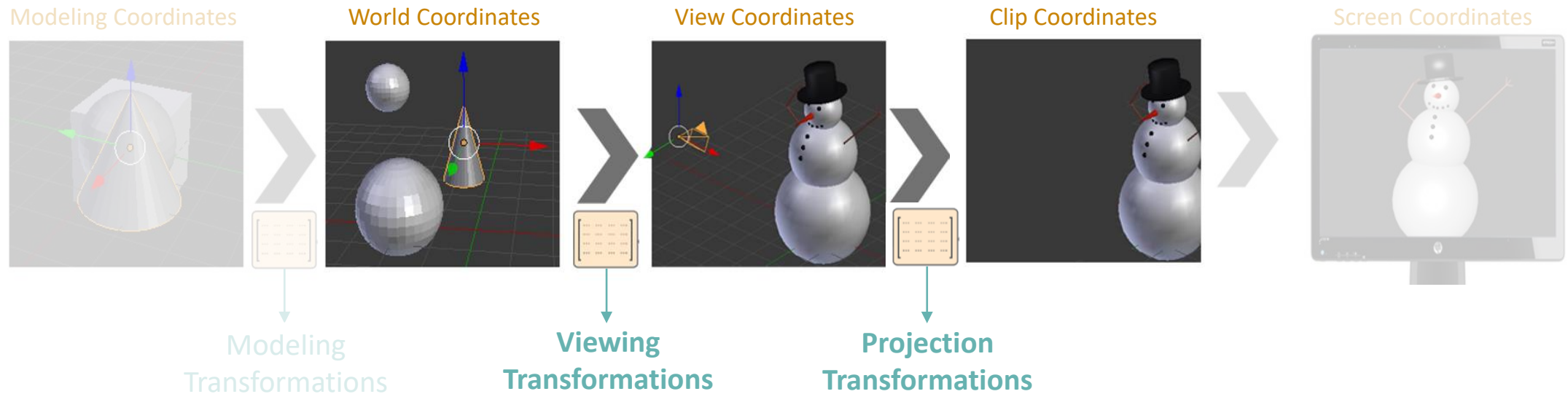
Computer Graphics

CPIT285

Dr. Rayed ALGhamdi

PREFACE

- In the previous chapter, we learn how to move from modeling coordinates to world coordinates using Modeling Transformations.
- In this chapter, let's learn how to move from world coordinated to view coordinates to clip coordinates using **Viewing & Projection Transformations**.



OpenGL 4x4 Matrix Modes

Model-View Mode

Modeling
Transforming

Viewing
Transforming

```
GL.glMatrixMode(GL.GL_MODELVIEW)
```

Projection Mode

Clipping

Viewport

Normalization

```
GL.glMatrixMode(GL.GL_PROJECTION)
```

PREFACE

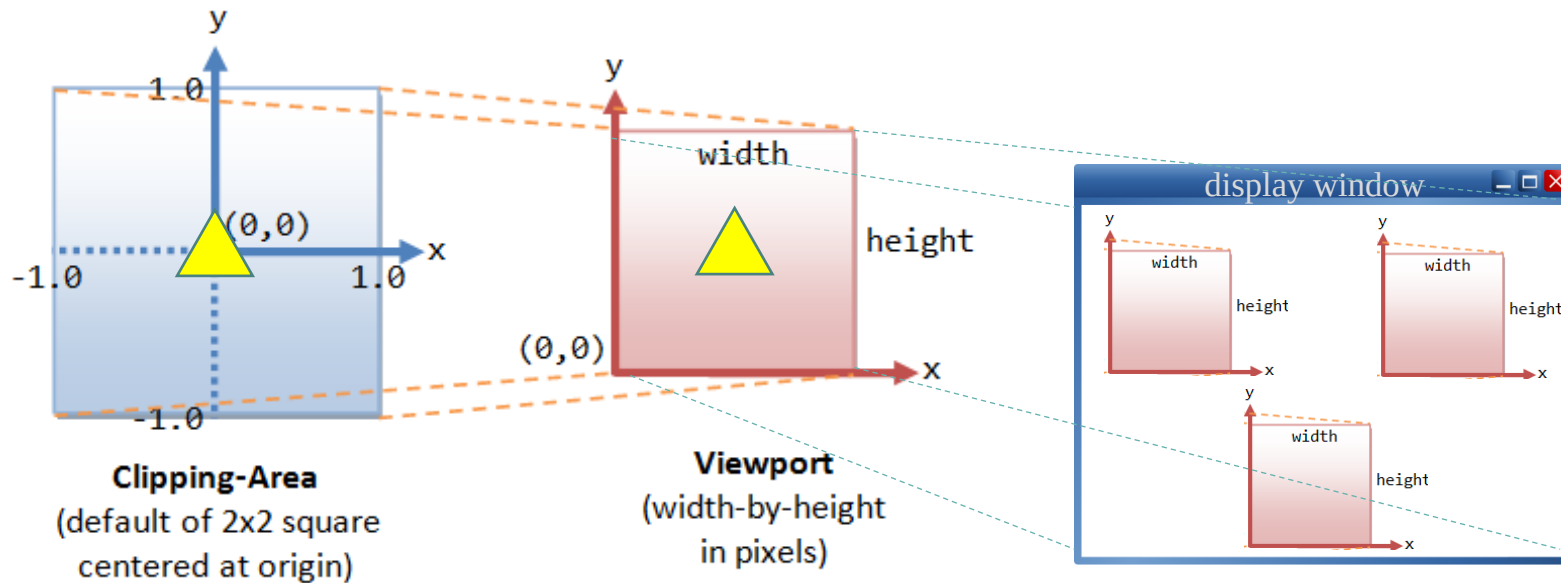
- 'Viewing Transformation' explains itself!
- It is simply how do you view your scene.
- We are often concerned with representing 3D scenes on a 2D surface. In order to generate an image of a 3D object on a 2D display, we must first project this object onto a projection plane and then display this projected image.
- 'Projection transformation' converts the viewing frustum into a cuboid shape. The near end of the viewing frustum is smaller than the far end, which has the effect of expanding objects that are near to the camera. This is how perspective is applied to the scene.

2D VIEWING TRANSFORMATION



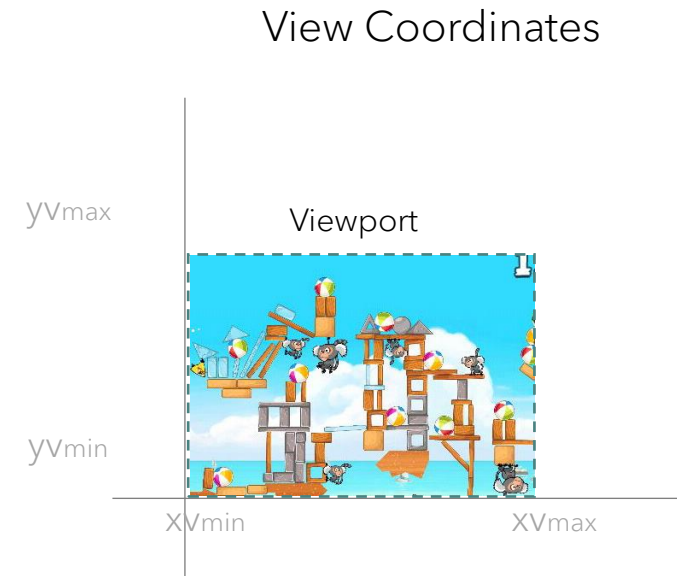
2D VIEWING TRANSFORMATIONS

- Unless the drawing space is defined, OpenGL by default defines the drawing area -1 to +1 for each axis that's called "clipping area"
- Whatever is drawn out of the clipping area cannot be viewed



2D VIEWING TRANSFORMATIONS

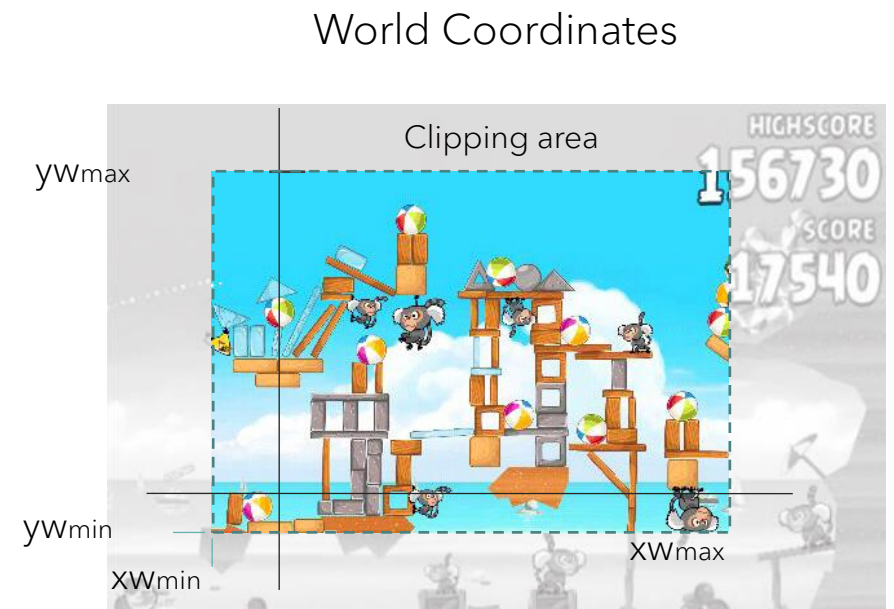
- Objects/draws within the clipping area are processed (using viewing transformations) to be viewed in a viewport.



2D VIEWING TRANSFORMATIONS

- The default clipping area is 2x2 (-1 to +1 for each axis).
- `glOrtho` function is used to size the clipping area.
- Whatever out the clipping area is discarded.

```
#Use the matrix in the projection mode
GL.glMatrixMode(GL.GL_PROJECTION)
#Matrix reset
GL.glLoadIdentity()
#Define the clipping area: (-x,x, -y,y, -z, z)
GL.glOrtho(-10.0, 10.0, -10.0, 10.0, 0.0, 0.0)
```

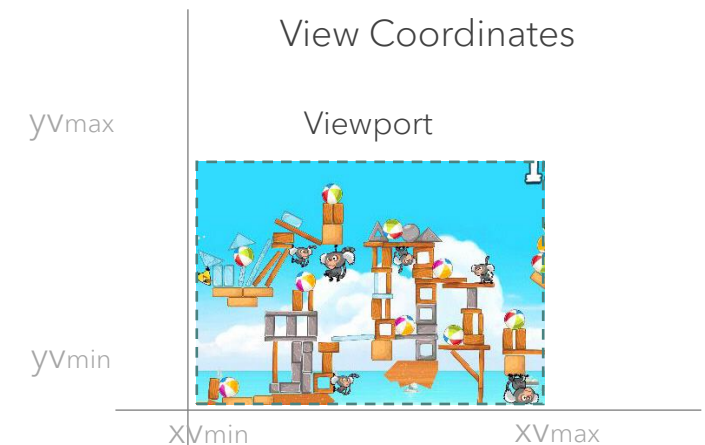


2D VIEWING TRANSFORMATIONS

- Viewport defines the display area on the display window. It's measured in pixels in screen coordinates.
- The clipping area is mapped to the viewport. glViewport function is used to configure the viewport.

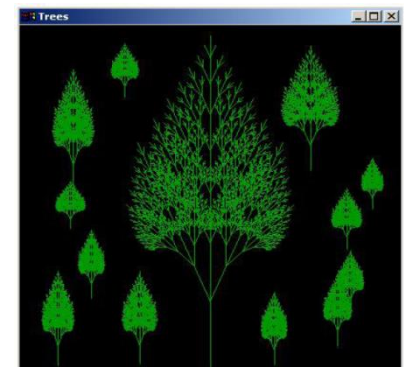
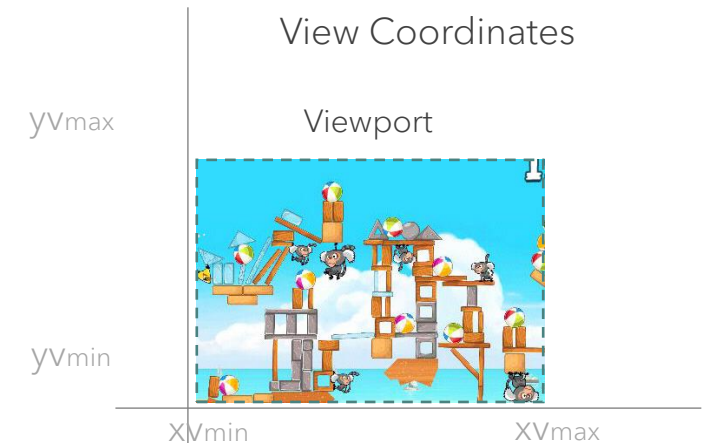
glViewport(xTopLeft,yTopLeft,width,height)

- When defining the viewport area, consider to be compatibility of aspect ratio with the width & high previously determined for the clipping area; If the area of the drawing is square (2x2), for example, we define the same aspect ratio for the viewport (eg: 100x100, 400x400, etc).
- Unless the viewport is defined, it takes the same values of the display window by default.



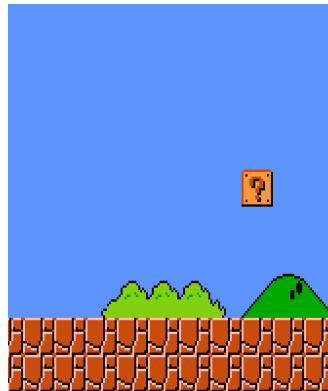
2D VIEWING TRANSFORMATIONS

- What's the point of viewport?
- Is not the display window enough?
 - Maintaining the object's appearance and their relative dimensions regardless of the changes of the display window diminutions.
 - Show more than one version of the same drawing in similar/different sizes, view...
 - It can be Useful in animation, how?...

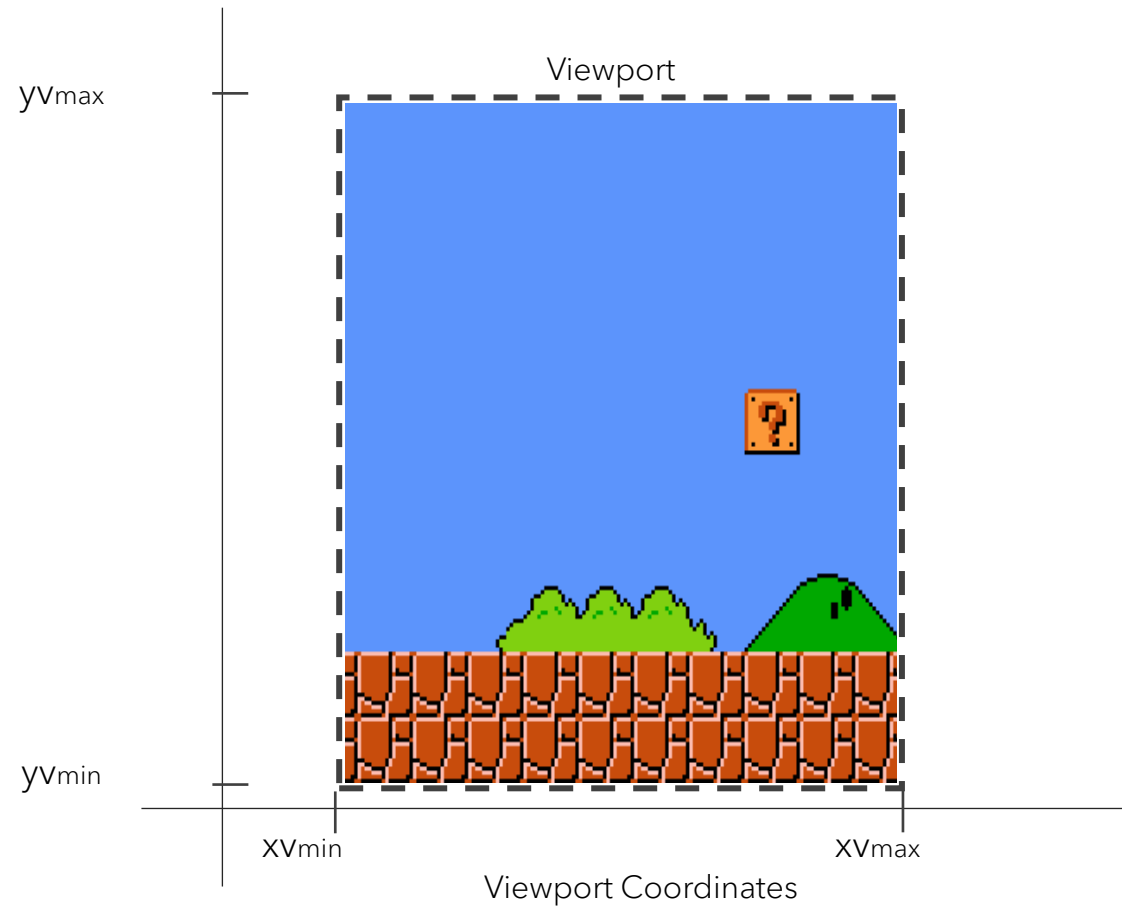


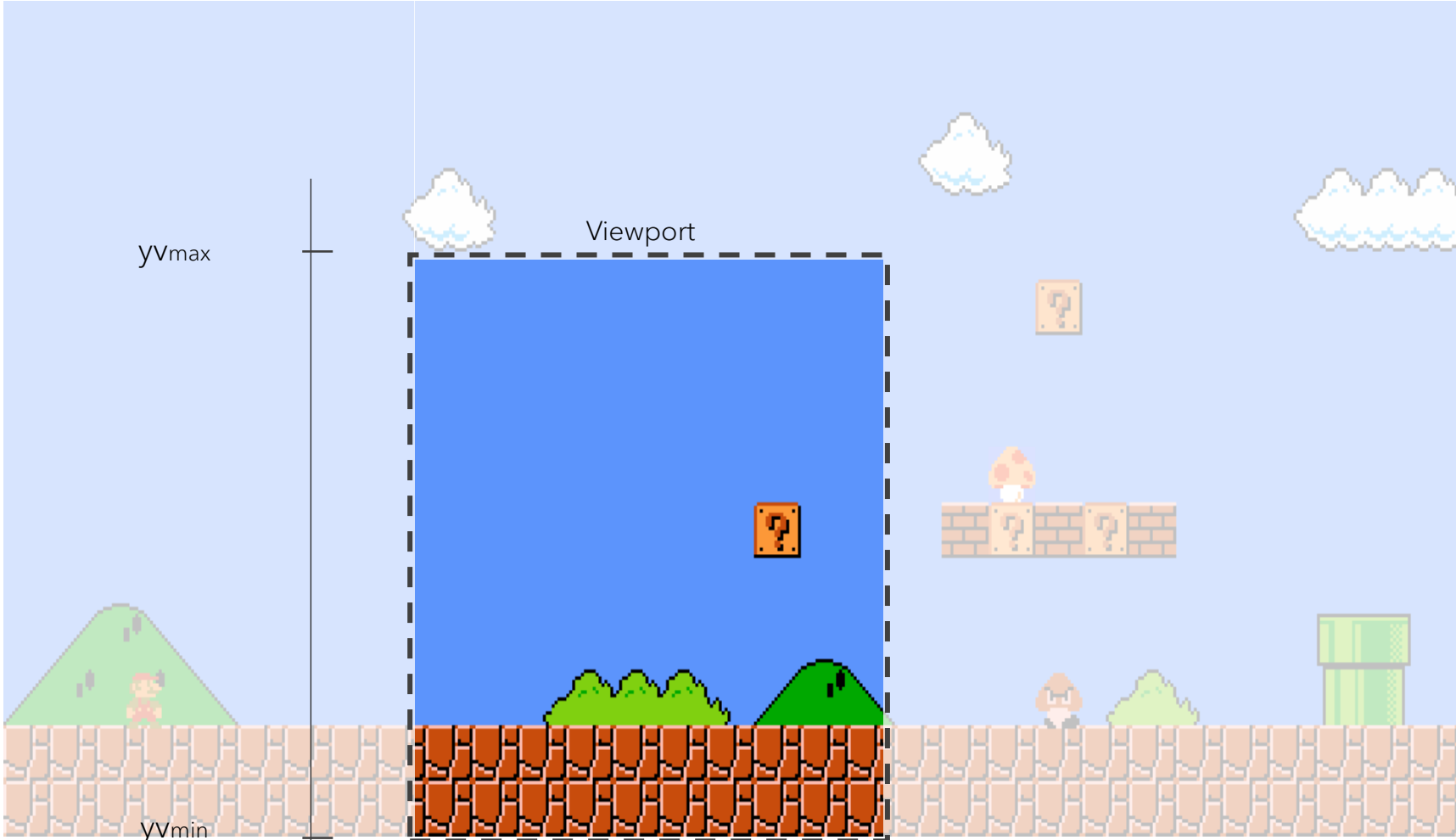
2D VIEWING TRANSFORMATIONS

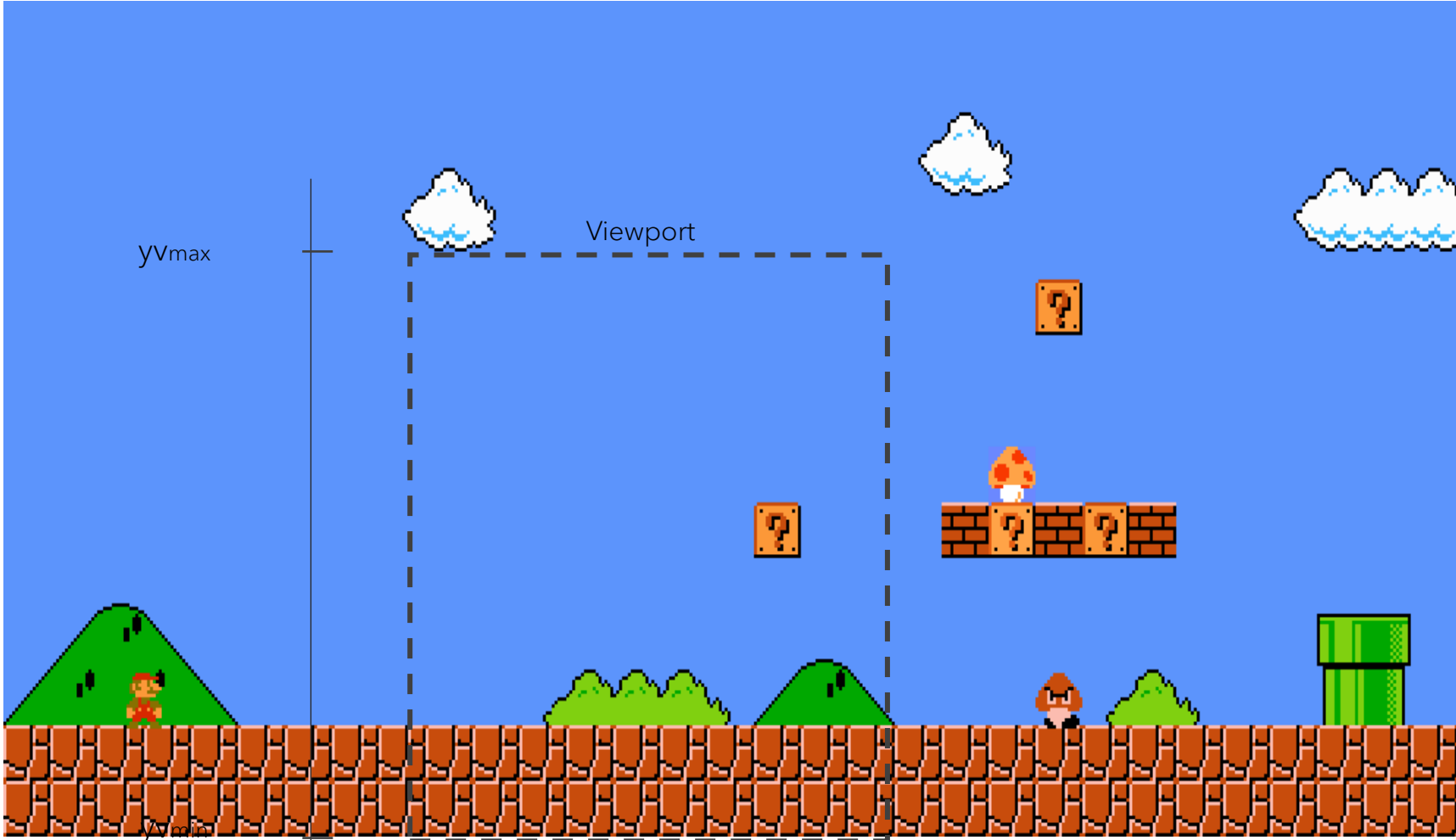
- Why all of these processes?
 - Does OpenGL produce only static graphics?!
 - What about computer games?
 - How do objects in a game move, interact, and be viewed?...



2D VIEWING TRANSFORMATIONS





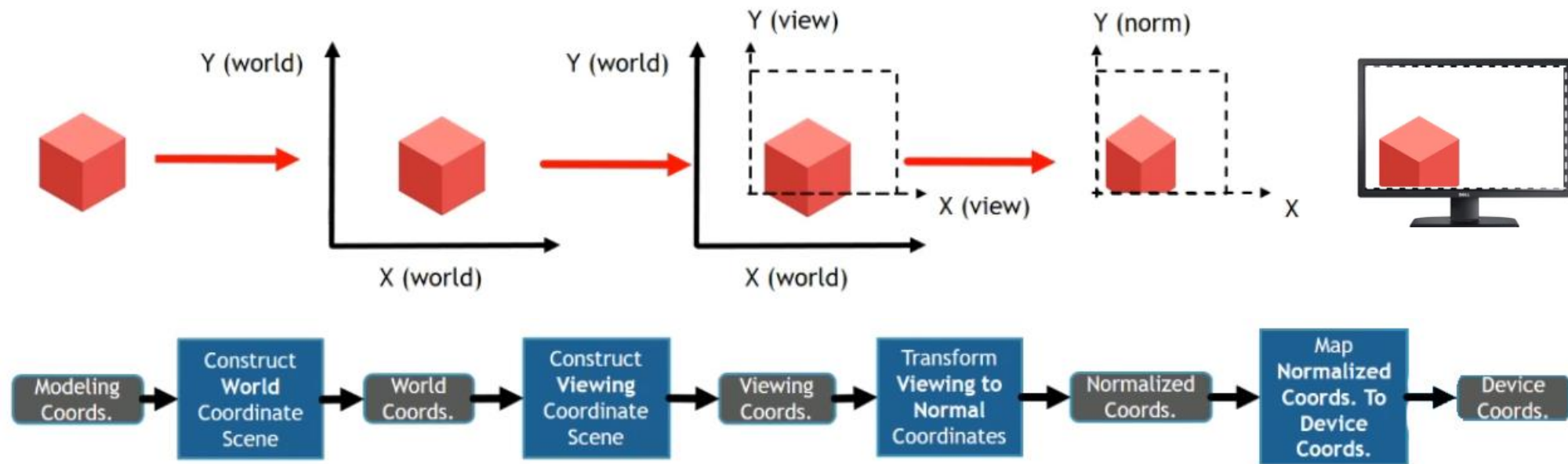




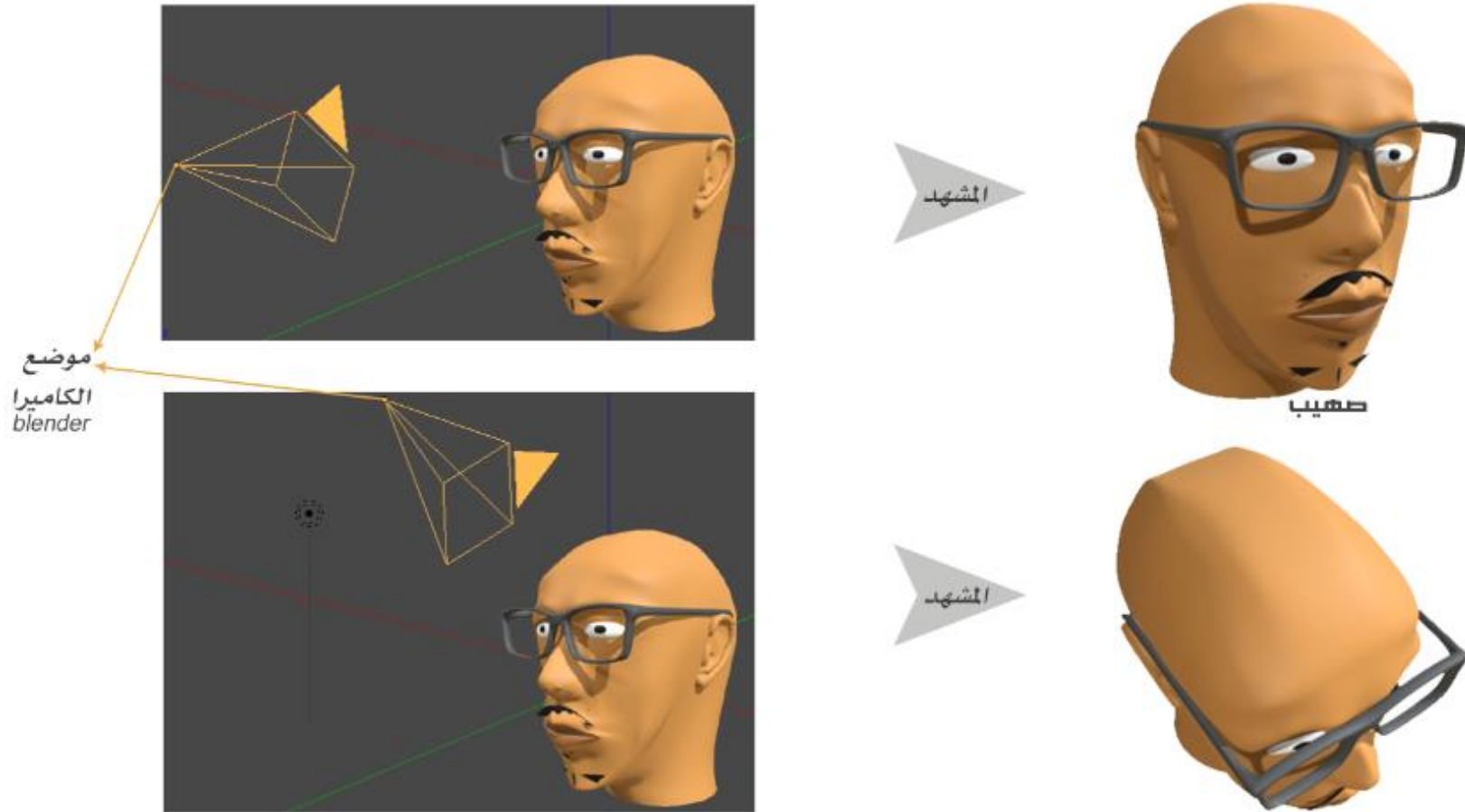
3D Viewing & Projection Transformations

3D TRANSFORMATIONS

- Unlike 2D graphics, we need to see stereoscopic objects in 3D.
- Several functions need to be used in order to control the view of 3D graphics.



3D TRANSFORMATIONS



3D VIEWING TRANSFORMATIONS

Viewing Transformation

- Unlike 2D
- We've learned in the head modeling assignment how to play with the camera to make the model is viewed from a specific angle.
- Viewing 3D graphics need to be coded in OpenGL
- We need to see the model from the top, down, right, left, front, or/and backside.
- `gluLookAt()` Function is used for this purpose
 - It takes 9 parameters



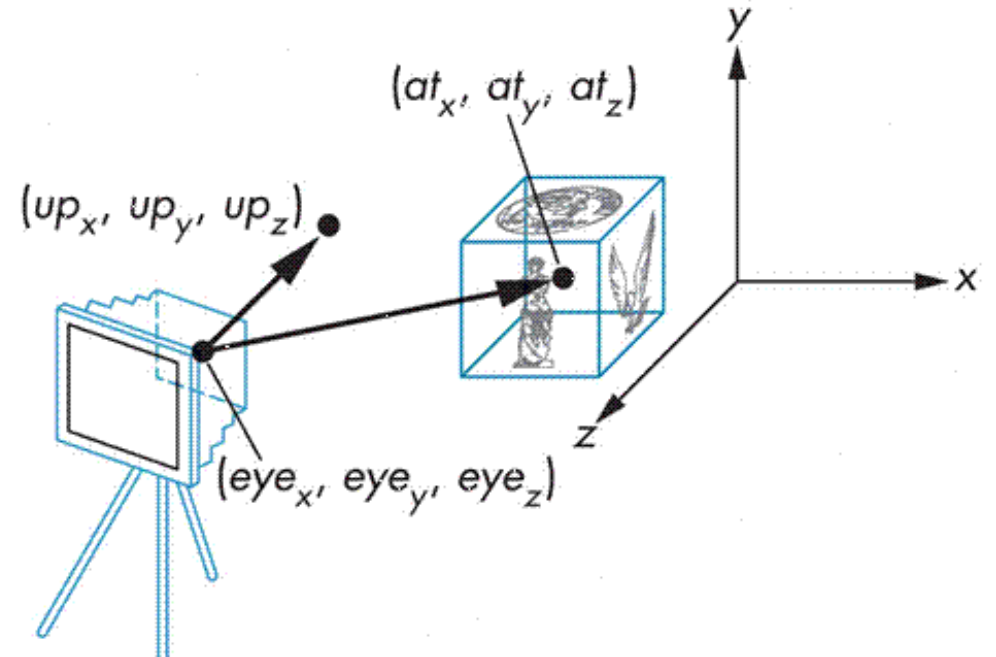
3D VIEWING TRANSFORMATIONS

Viewing Transformation

- `gluLookAt();`

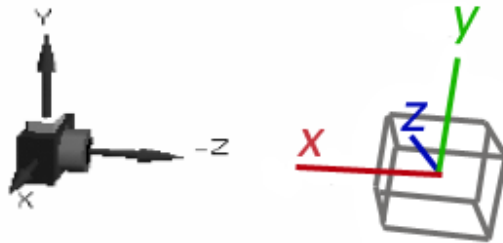
`gluLookAt (eyex, eyey, eyez, atx, aty, atz, upx, upy, upz);`

- The first three define eye position
- The following three position the camera
- The last three define looking position (up/down)



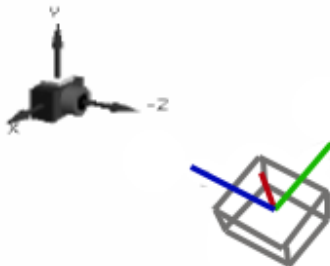
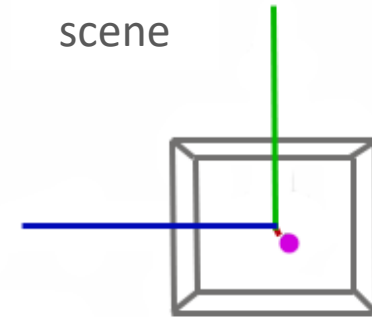
3D VIEWING TRANSFORMATIONS

Viewing angle

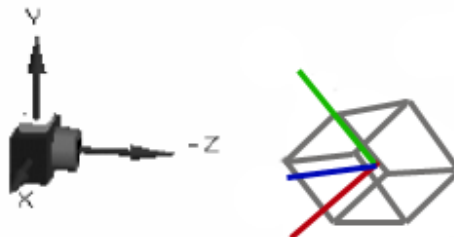
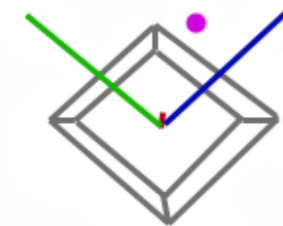


```
glutLookAt (4,0,0, 0,0,0, 0,1,0);
```

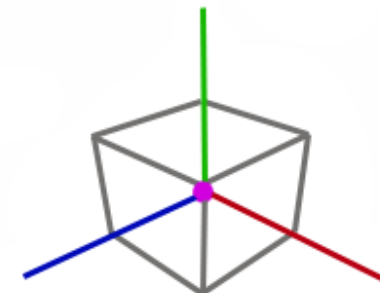
scene



```
glutLookAt (0,0,4, 0,1,1, 1,1,0);
```



```
glutLookAt (2,2,2, 0,0,0, 0,1,0);
```



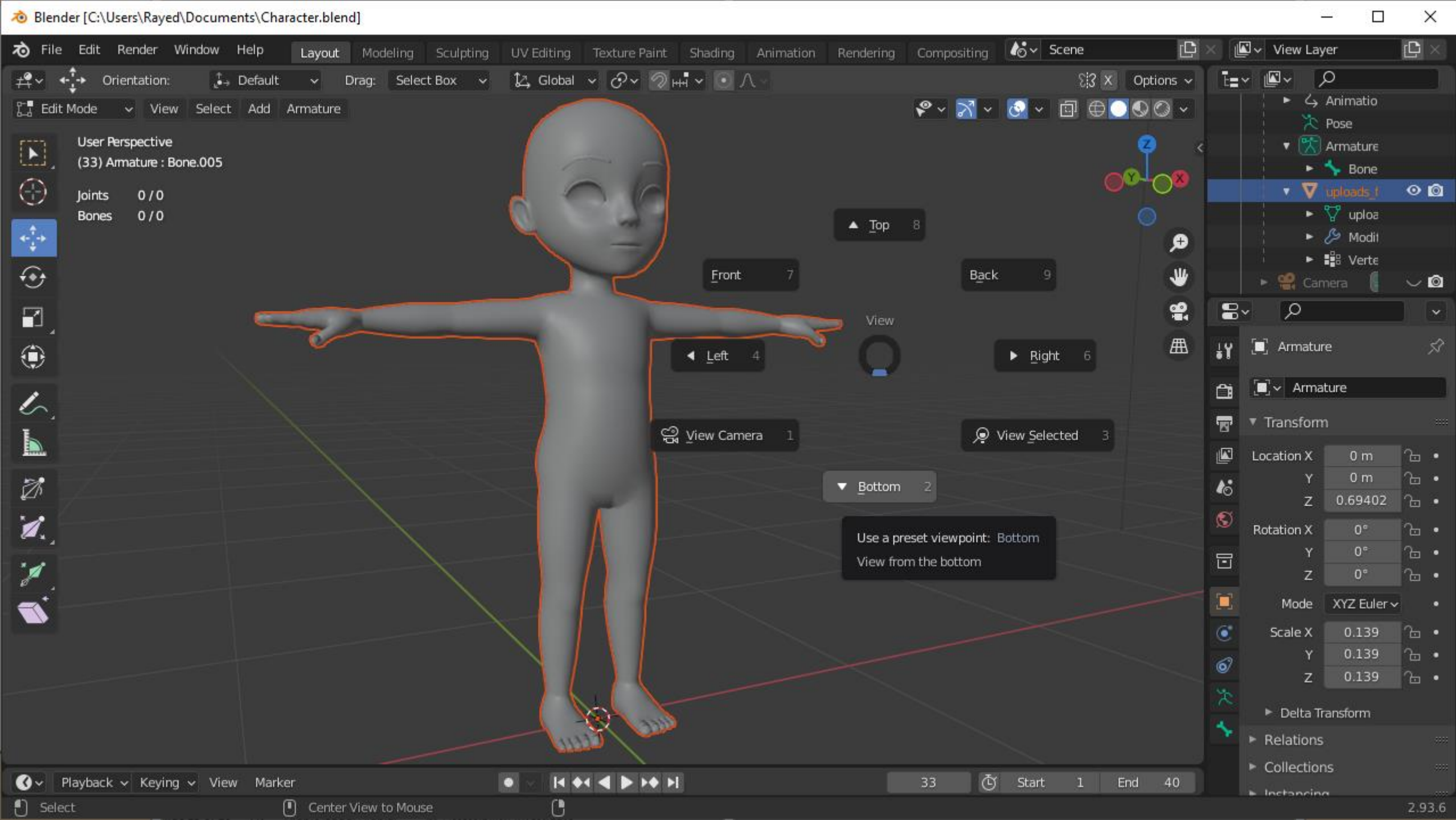
3D VIEWING TRANSFORMATIONS

Viewing Transformation

- If the function of `gluLookAt()` parameters are not defined, the following values are given by default

```
glMatrixMode(GL_MODELVIEW);  
glLoadIdentity();  
gluLookAt(eyex, eyey, eyez, atx, aty, atz, upx, upy, upz);
```

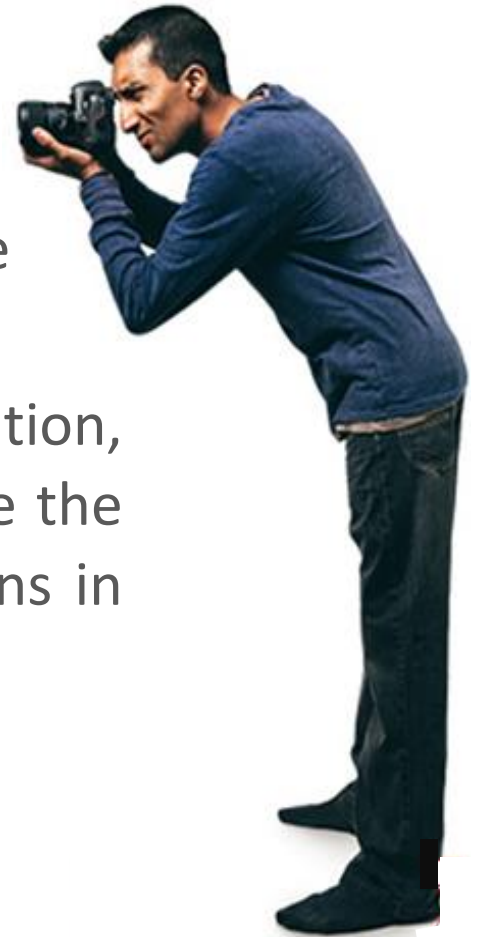
- `eyex, eyey, eyez = (0, 0, 0)`
- `atx, aty, atz = (0, 0, -1)`
- `upx, upy, upz = (0, 1, 0)`



3D PROJECTION TRANSFORMATIONS

Projection

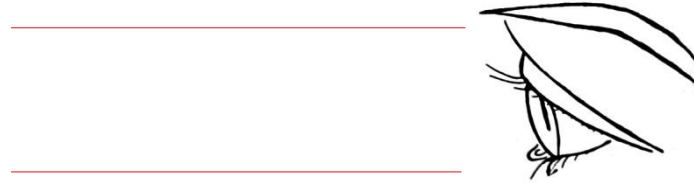
- In determining a viewing angle, eye position and side of looking are defined to simulate what a photographer does when taking a picture with a camera.
- When a photographer determines where he stands and takes a position, he starts controlling the camera lens (zooming in/out) to determine the camera shoot zone. The shooting zone is determined by projections in OpenGL.
- Anything is out of the shooting zone/projection is not visible.



3D PROJECTION TRANSFORMATIONS

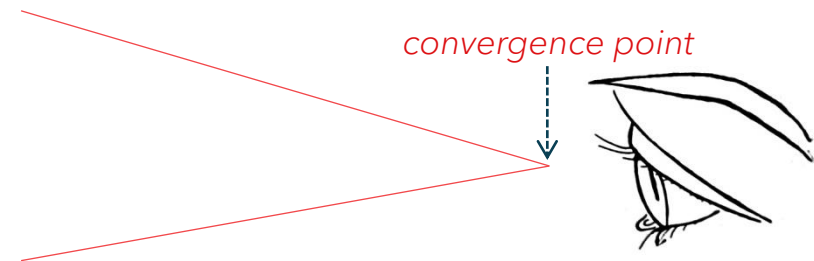
- Orthographic Projection

It has no convergence point in which the projection lines are perpendicular to the view plane.



- Perspective Projection

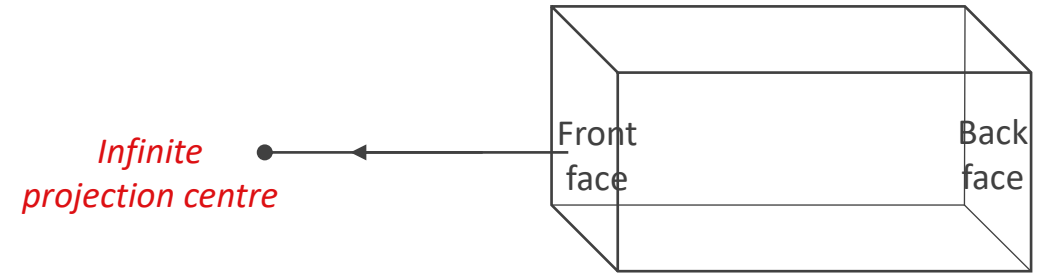
It starts at a convergence point. When a scene is viewed in faraway objects appear small relative to nearer objects.



3D PROJECTION TRANSFORMATIONS

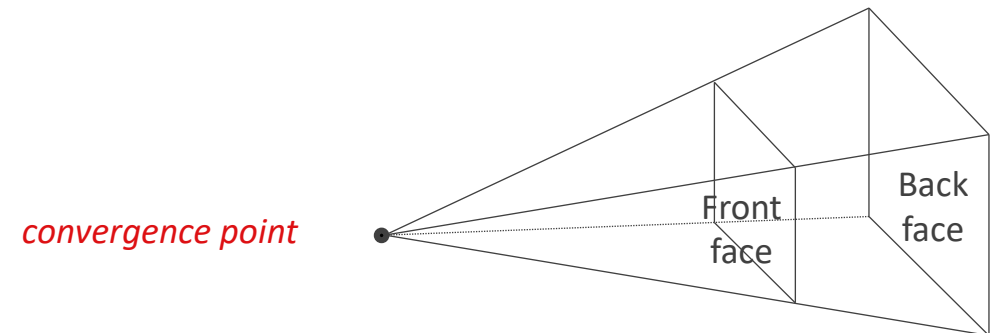
○ Orthographic Projection

Vision zone is all that lies between the front and back faces of the rectangular.



○ Perspective Projection

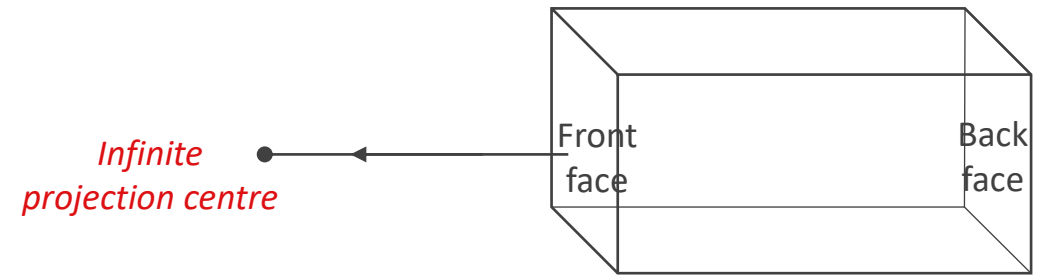
Vision zone is all that lies between the front and back faces of the pyramid.



3D PROJECTION TRANSFORMATIONS

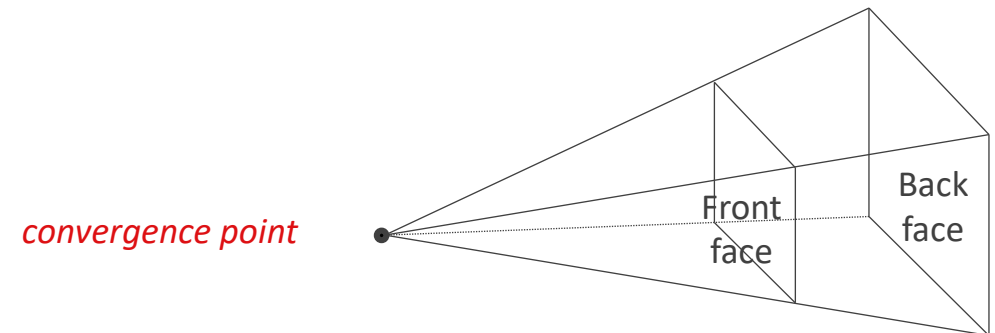
○ Orthographic Projection

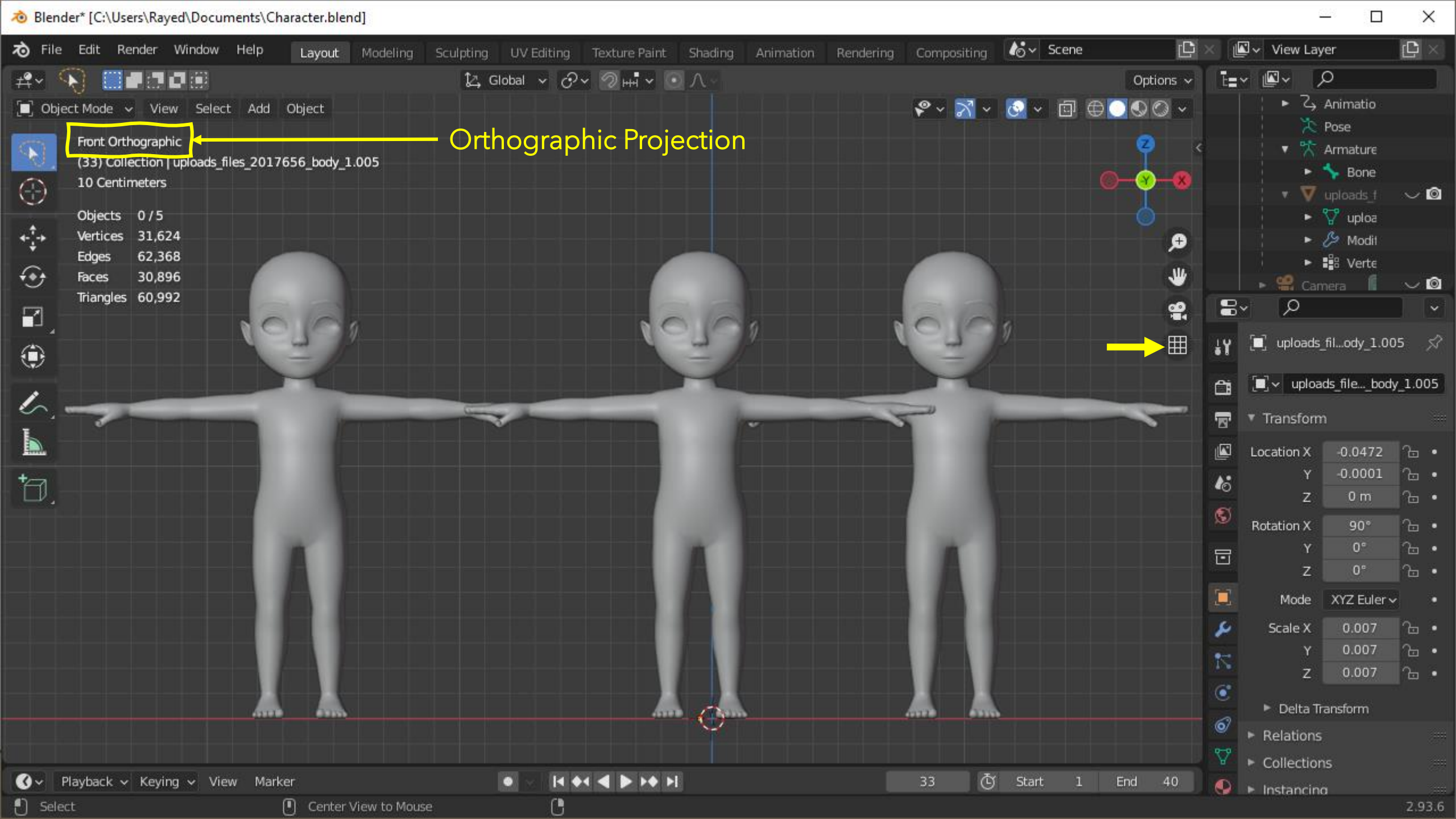
Vision zone is all that lies between the front and back faces of the rectangular.



○ Perspective Projection

Vision zone is all that lies between the front and back faces of the pyramid.





Front Orthographic

Orthographic Projection

(33) Collection | uploads_files_2017656_body_1.005

10 Centimeters

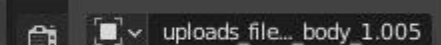
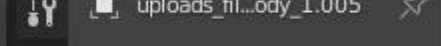
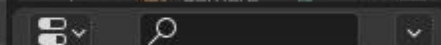
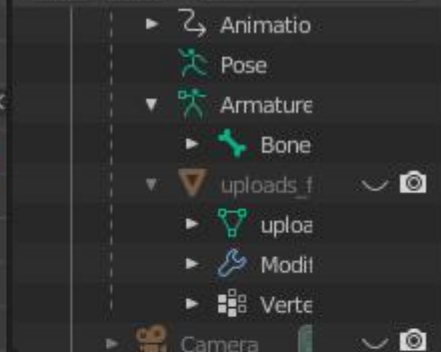
Objects 0 / 5

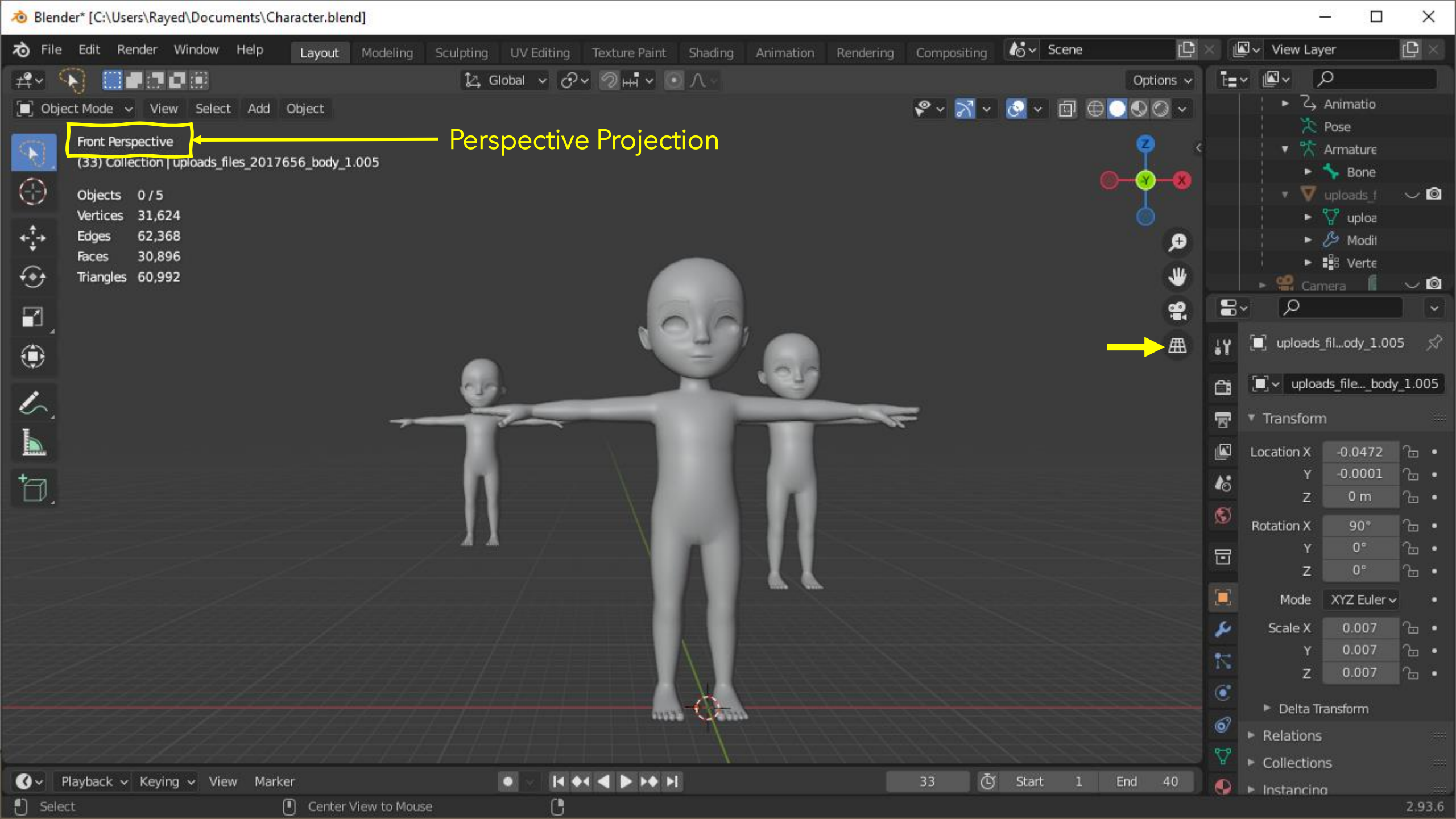
Vertices 31,624

Edges 62,368

Faces 30,896

Triangles 60,992





Front Perspective

Perspective Projection

uploads_file...ody_1.005

uploads_file..._body_1.005

Transform

Location X -0.0472

Y -0.0001

Z 0 m

Rotation X 90°

Y 0°

Z 0°

Mode XYZ Euler

Scale X 0.007

Y 0.007

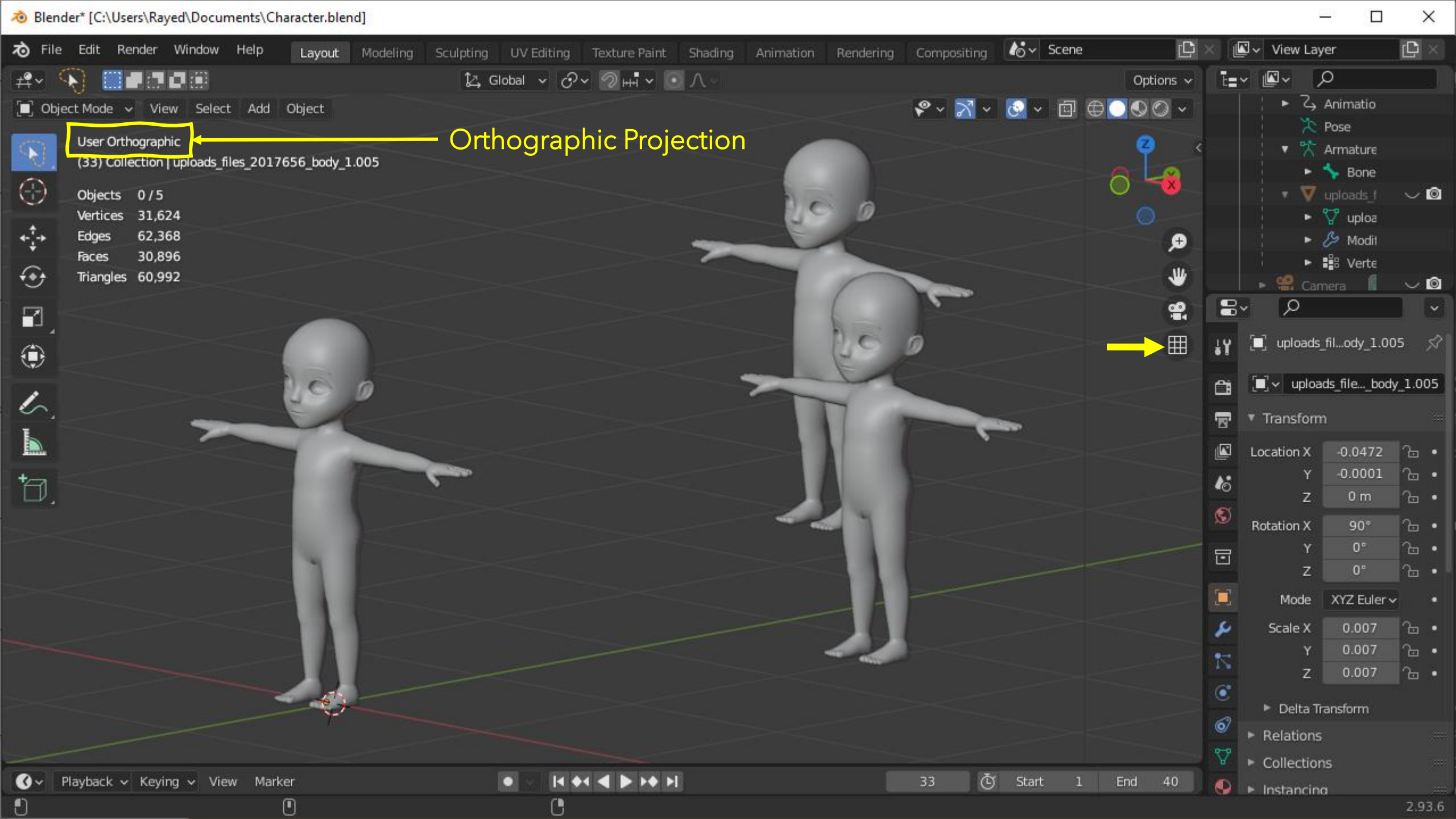
Z 0.007

Delta Transform

Relations

Collections

Instancing



User Orthographic

Orthographic Projection

uploads_file...ody_1.005

uploads_file..._body_1.005

Transform

Location X -0.0472

Y -0.0001

Z 0 m

Rotation X 90°

Y 0°

Z 0°

Mode XYZ Euler

Scale X 0.007

Y 0.007

Z 0.007

Delta Transform

Relations

Collections

Instancing



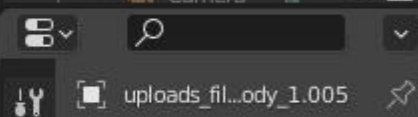
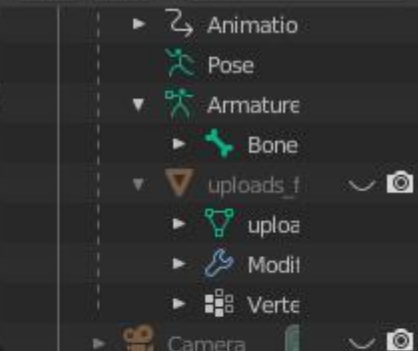
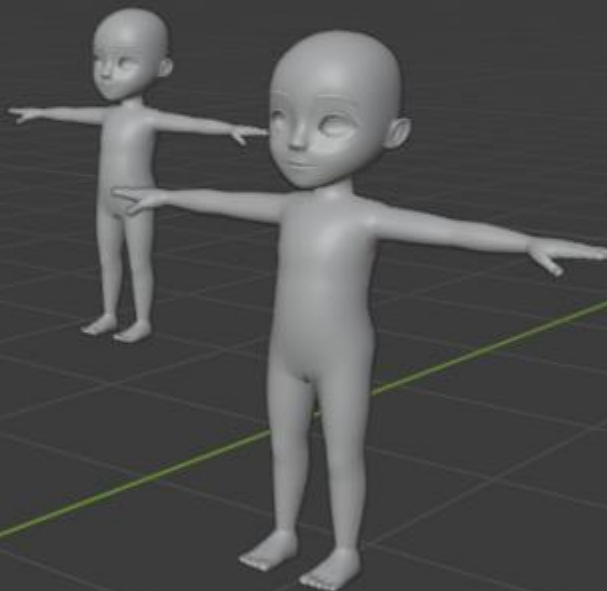
Object Mode View Select Add Object

User Perspective

Perspective Projection

(33) Collection | uploads_files_2017656_body_1.005

Objects 0 / 5
Vertices 31,624
Edges 62,368
Faces 30,896
Triangles 60,992



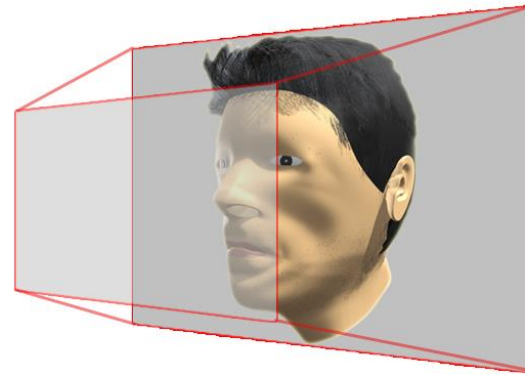
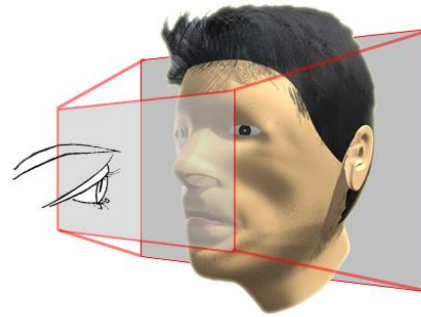
Physics Properties

Collections

Instancing

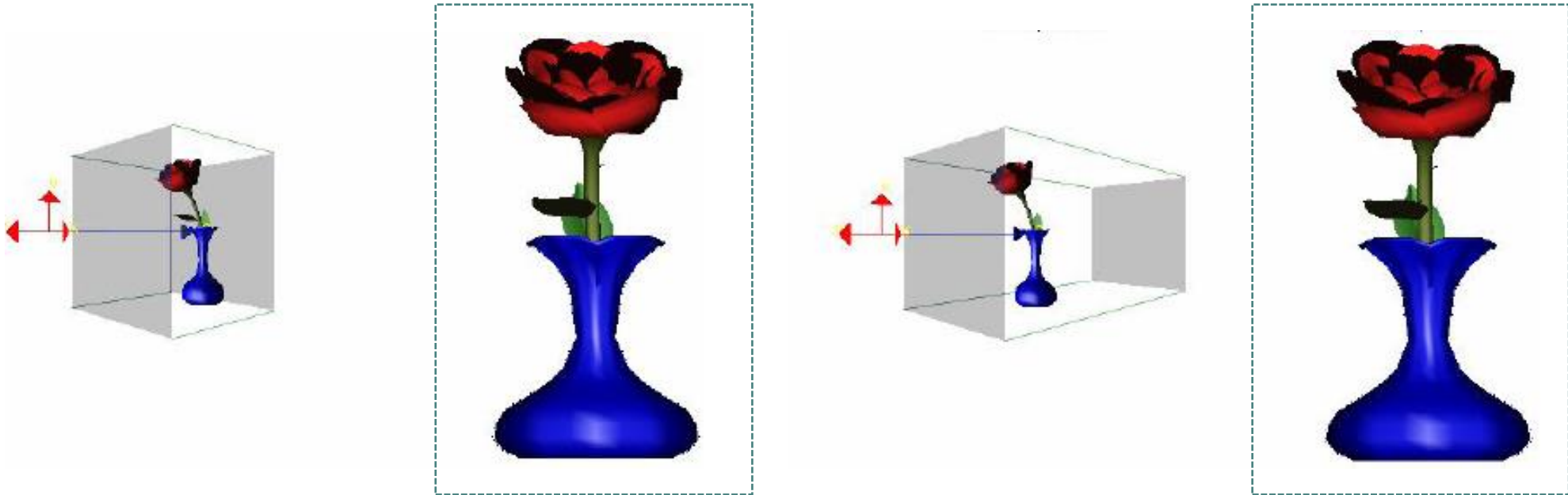
3D PROJECTION TRANSFORMATIONS

- The difference between the two types of projection appears in the zoom in/out of the camera/eye.
- In **perspective projection** zoom in/out affects an object sizing



3D PROJECTION TRANSFORMATIONS

- In **orthographic projection**, zoom in/out has no effect on the size of an object.



3D PROJECTION TRANSFORMATIONS

Projection

- In the perspective projection, objects which are far away are smaller than those nearby.
- In the orthographic projection, all objects appear at the same scale.
- Since some prefer one over the other, both options are available.
- Perspective projection gives more depth information and is often easier to view because you use perspective views in real life.
- Orthographic projection makes it much easier to compare two parts of the molecule, as there is no question about how the viewpoint may affect the perception of distance.

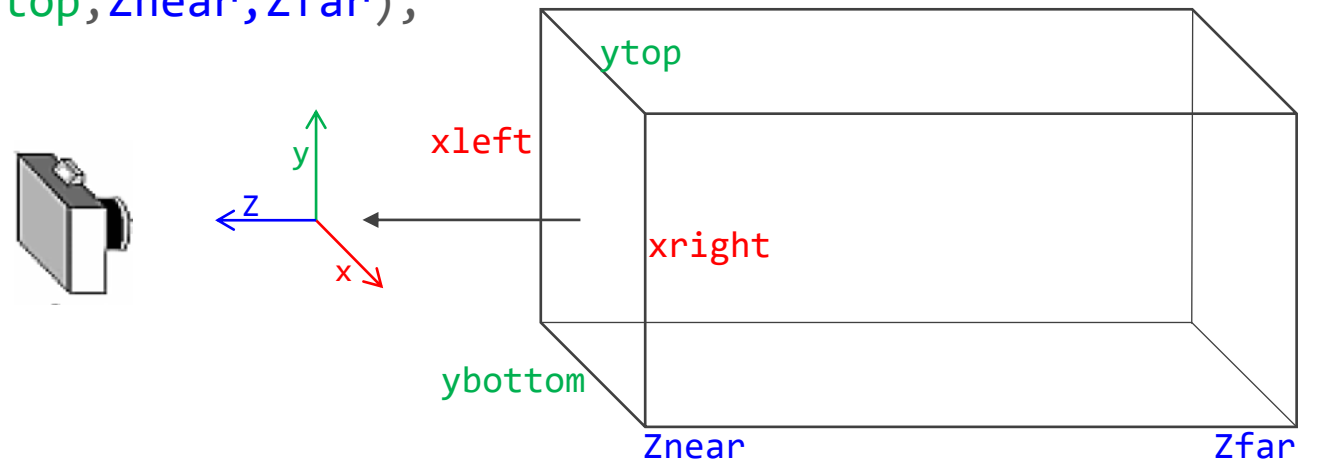
3D PROJECTION TRANSFORMATIONS

- If the **orthographic projection** isn't defined, it takes default values:
-1 to +1 for each axis.

```
glMatrixMode (GL_PROJECTION);
```

```
glLoadIdentity();
```

```
glOrtho (xleft,xright,ybottom,ytop,Znear,Zfar);
```



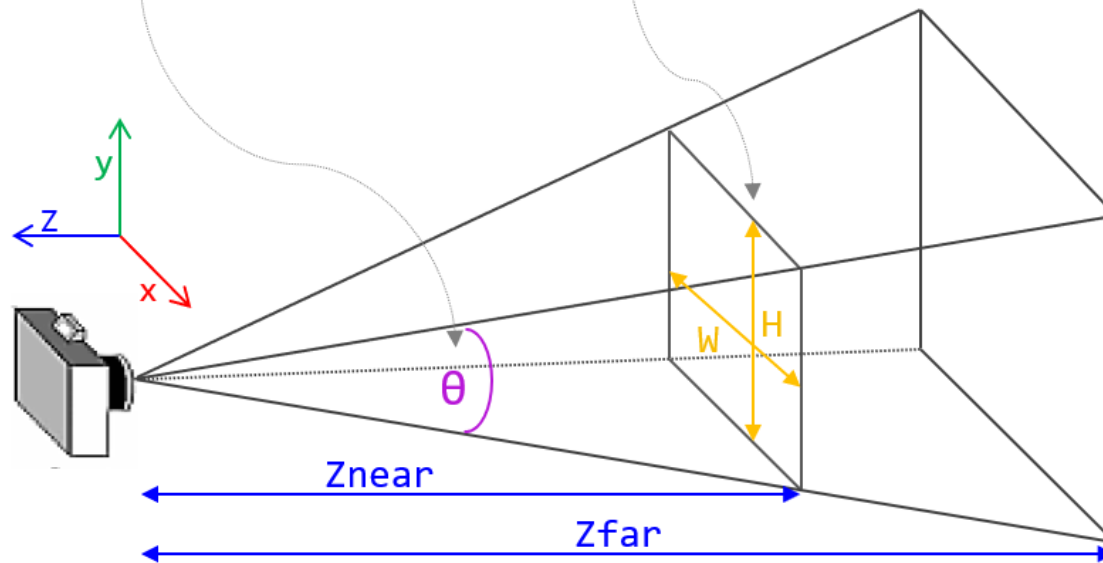
3D PROJECTION TRANSFORMATIONS

```
gluPerspective (fov, aspect, Znear, Zfar);
```

angular aperture / size of the lens

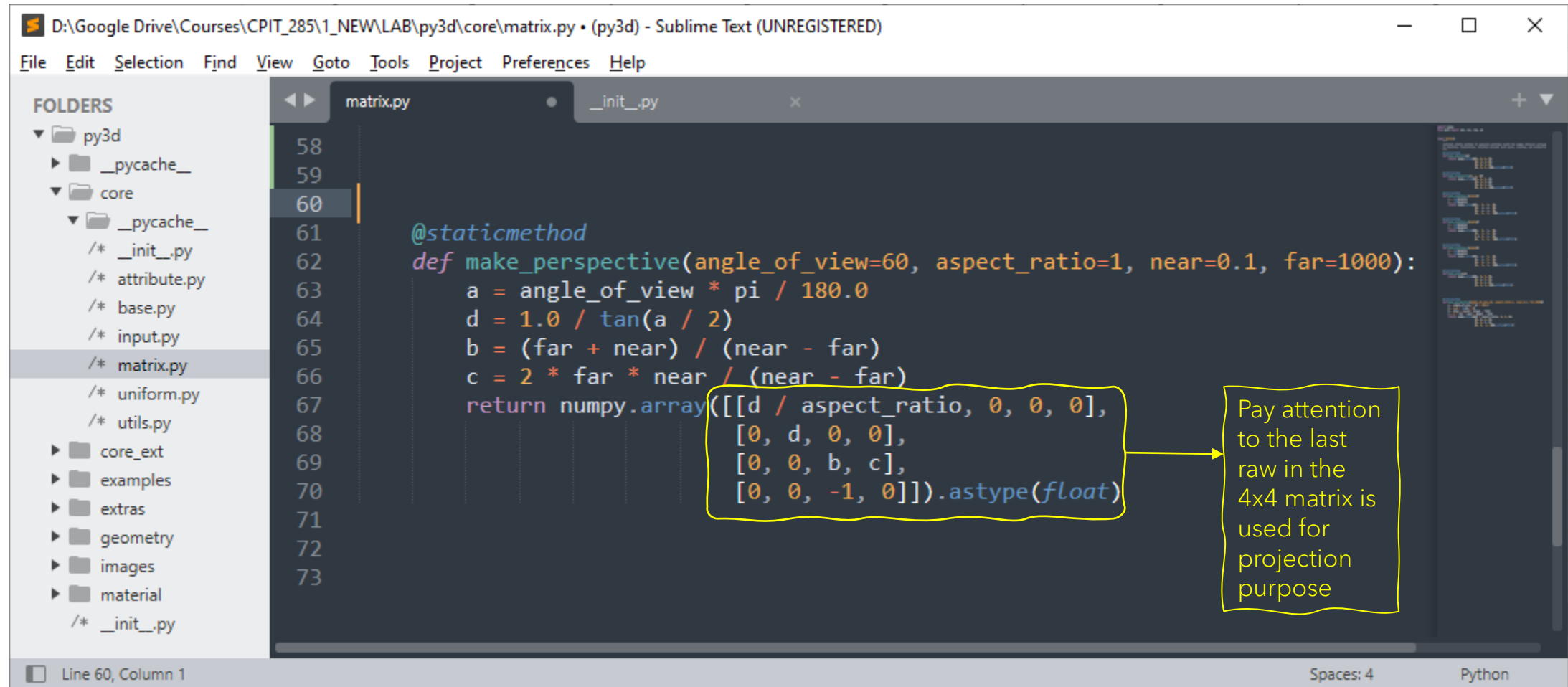
θ

Aspect of the window width/high



- The **perspective projection** can be defined using `gluPerspective` function

3D PROJECTION TRANSFORMATIONS



```
D:\Google Drive\Courses\CPIT_285\1_NEW\LAB\py3d\core\matrix.py • (py3d) - Sublime Text (UNREGISTERED)
File Edit Selection Find View Goto Tools Project Preferences Help

FOLDERS
▼ py3d
  ► __pycache__
  ▼ core
    ▼ __pycache__
      /* __init__.py
      /* attribute.py
      /* base.py
      /* input.py
      /* matrix.py
      /* uniform.py
      /* utils.py
    ► core_ext
    ► examples
    ► extras
    ► geometry
    ► images
    ► material
    /* __init__.py

matrix.py
__init__.py

58
59
60
61 @staticmethod
62 def make_perspective(angle_of_view=60, aspect_ratio=1, near=0.1, far=1000):
63     a = angle_of_view * pi / 180.0
64     d = 1.0 / tan(a / 2)
65     b = (far + near) / (near - far)
66     c = 2 * far * near / (near - far)
67     return numpy.array([[d / aspect_ratio, 0, 0, 0],
68                         [0, d, 0, 0],
69                         [0, 0, b, c],
70                         [0, 0, -1, 0]]).astype(float)
71
72
73

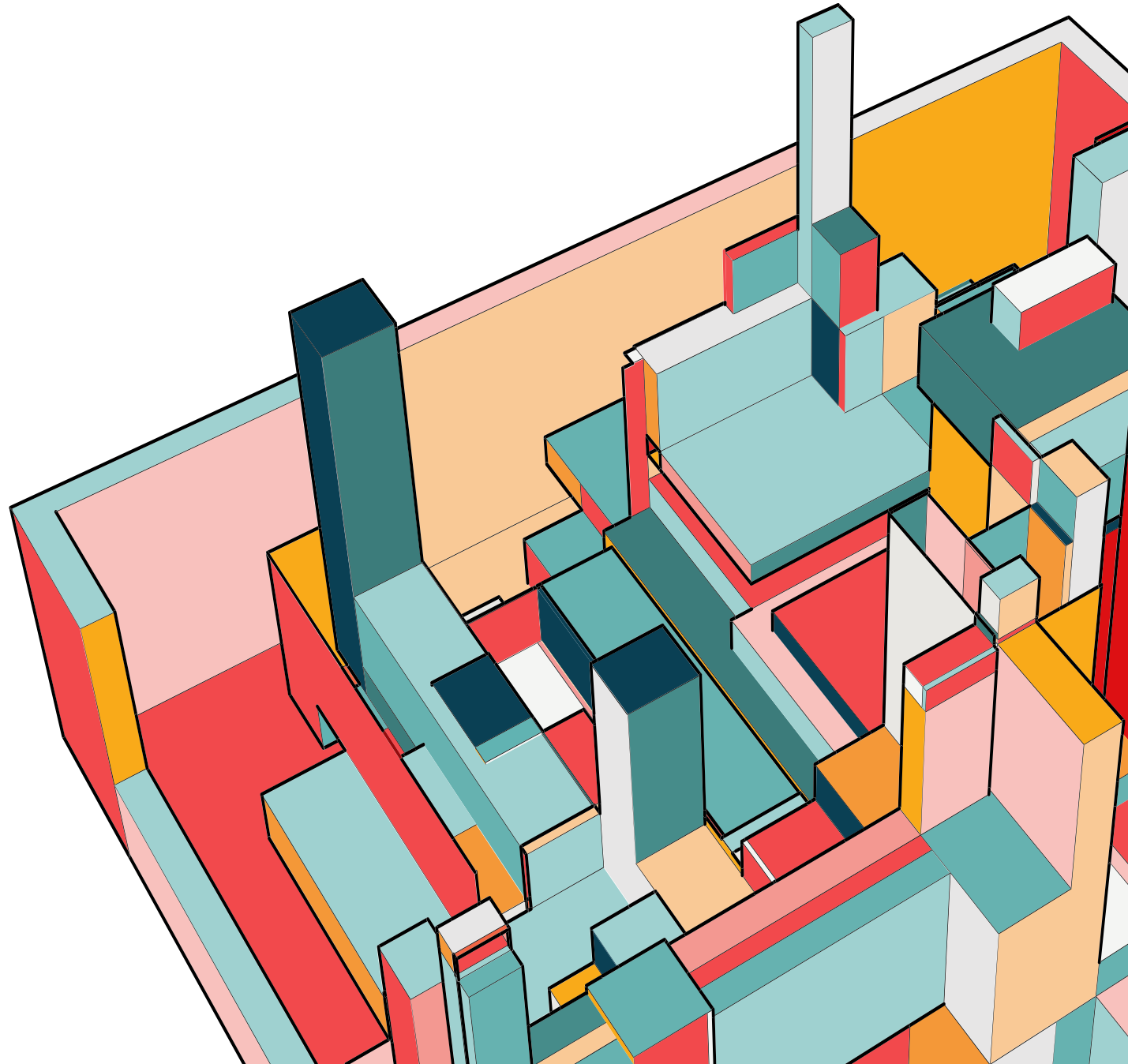
Pay attention to the last row in the 4x4 matrix is used for projection purpose

Line 60, Column 1 Spaces: 4 Python
```

100% CONFIDENT

Your journey in this course is
enjoyable and beneficial!

Enjoy learning CPIT285!



THANK YOU

Dr. Rayed ALGhamdi

+966-12-640-0000 ex:75560

raalghamdi8@kau.edu.sa

www.raalghamdi8.kau.edu.sa

